

DriveMA: Driving Vision-Language-Action Models with verifiable Meta-Actions

Weicheng Zheng^{1,2} Yixin Huang^{1,3} Qiao Sun¹ Derun Li¹ Hang Zhao^{1,2*}

¹Shanghai Qi Zhi Institute

²IIS Tsinghua University

³Tongji University

Abstract: Driving Vision-Language-Action Models (Driving VLAs) aim to use language to improve end-to-end planning, but the language-action gap limits this promise. We propose **DriveMA**, a Driving VLA framework built on verifiable meta-actions, which summarize future ego motion into compact language-domain intentions and can be constructed from expert trajectories with a trajectory-grounded annotation pipeline and can be verified against generated trajectories through rule-based projection. DriveMA exploits this verifiability with action-centric supervised training and a data-efficient turn-level credit assignment reinforcement learning framework, explicitly aligning high-level decisions with low-level trajectory planning through dense rewards and precise credit assignment. DriveMA sets a new state of the art on the Waymo Open Dataset Vision-based E2E Driving, achieving a Rater Feedback Score of **8.060** with a 2B model and further improving it to **8.079** with a 4B model; it also obtains competitive closed-loop planning performance on NAVSIM. These results show that even a simple meta-action interface can achieve state-of-the-art planning when made verifiable and optimized for language-action alignment. Code, data, and models are available at <https://tsinghua-mars-lab.github.io/DriveMA>.

Keywords: Autonomous Driving, Vision-Language-Action, Meta-Action

1 Introduction

Driving Vision-Language-Action Models (Driving VLAs) have recently emerged as a promising paradigm for end-to-end autonomous driving [1, 2, 3, 4]. By introducing language into the perception-to-action pipeline, these models aim to leverage semantic knowledge to improve downstream planning. Ideally, language should not merely describe the scene, but should expose high-level driving intentions that guide low-level trajectory generation. However, the persistent language-action gap in Driving VLAs means that plausible language-side decisions may not be faithfully executed by predicted trajectories, preventing language from becoming a truly actionable planning interface [5, 6]. Fig. 1 illustrates a concrete language-action mismatch: despite predicting acceleration at a green light, an SFT model remains nearly stationary under a near-static motion history.



Figure 1: **Motivating example of the language-action gap.** At a green light with near-static motion history, the SFT-only DriveMA predicts a plausible high-level decision to accelerate, but generates an almost stationary trajectory, revealing a language-action mismatch. With our RL-based explicit alignment, the trajectory better executes the predicted acceleration decision.

*Corresponding author: hangzhao@mail.tsinghua.edu.cn

We argue that bridging this gap requires the intermediate language interface to be *verifiable*. A verifiable interface allows high-level language decisions to be supervised from expert behavior, checked against generated trajectories, and optimized through explicit alignment. In this paper, we study *meta-action* as a simple and effective instance of such a verifiable language interface. A meta-action compactly represents the planning-relevant intent of future ego motion, such as accelerating, decelerating, turning, or changing lanes. More importantly, trajectories can be projected back into a verifiable action space, making it possible to check whether the planned motion is consistent with the stated high-level decision. Although other forms of verifiable language interfaces may also be possible, this work focuses on how far a simple meta-action interface can go when its verifiability is fully exploited.

To this end, we propose **DriveMA**, a Driving VLA that instantiates meta-action as a simple verifiable language interface between visual inputs and trajectory generation. DriveMA exploits this interface through three components: trajectory-grounded meta-action annotation for scalable supervision, action-centric pretraining for driving-domain decision learning, and turn-level credit assignment RL for explicit language-action alignment with dense reward and precise credit assignment. Together, these components strengthen driving-domain decision knowledge while exploiting meta-action verifiability to produce trajectories that better reflect predicted decision, as illustrated in Fig. 1.

Experiments demonstrate that this simple verifiable-interface-centered design is highly effective. On the Waymo Open Dataset for Vision-Based End-to-End Driving (WOD-E2E) [7], DriveMA already achieves a new state of the art with a 2B model, reaching a **Rater Feedback Score (RFS) of 8.060**, while its 4B version further improves the state of the art to **8.079**. On NAVSIM [8], DriveMA obtains competitive closed-loop planning performance among existing Driving VLAs. Ablations further show that action-centric pretraining and turn-level credit assignment RL effectively align language decisions with trajectory planning. Overall, these results show that the key value of DriveMA lies not in introducing meta-actions as a new language interface, but in fully exploiting their verifiability to turn a simple language interface into an effective mechanism for language-action alignment. Our main contributions are summarized as follows:

- We propose **DriveMA**, a Driving VLA framework that instantiates meta-actions as a simple verifiable intermediate language interface and strengthens high-level decision learning through action-centric pretraining.
- We introduce a highly data-efficient turn-level credit assignment RL framework for explicit language-action alignment. It exploits meta-action verifiability to construct dense rewards and assigns them precisely across generation tokens.
- DriveMA sets a new state of the art on WOD-E2E already with a 2B model, reaching an RFS of **8.060**, and further pushes the record to **8.079** with its 4B version. It also obtains competitive closed-loop planning performance on NAVSIM.

2 Related Work

2.1 Language Interfaces for Driving VLAs

Recent language-based driving systems introduce natural language into end-to-end autonomous driving for instruction following, scene understanding, decision grounding, and interpretability [1, 2]. Many Driving VLA methods use language reasoning as the intermediate interface between visual inputs and actions [9, 10, 11, 4, 12]. For example, DriveVLM [3] decomposes driving into scene description, scene analysis, and hierarchical planning, while AutoVLA [4] unifies reasoning and action generation in an autoregressive framework with adaptive thinking modes. These works show that language can expose useful intermediate states before trajectory generation. Meanwhile, NoRD studies a complementary direction by reducing dense reasoning annotations and language overhead for efficient Driving VLAs [13]. Our work focuses on the high-level decision interface that often appears near the end of these language-based pipelines, such as driving intentions, maneuvers, or meta-action-like commands. DriveMA therefore isolates meta-action as a simple verifi-

able language interface and studies how to fully exploit it for supervision, reward design, and credit assignment.

2.2 Language-Action Alignment

Language-action alignment is a central challenge for applying VLA models to autonomous driving [14, 5, 15, 6]. SimLingo [5] aligns language and actions through multi-task training and Action Dreaming, grounding vision-language understanding in closed-loop driving behaviors. LinkVLA [15] improves language-action coupling by unifying language and action tokens in a shared token space and introducing an action-understanding objective to learn bidirectional trajectory–language mappings. Drive-R1 [6] mitigates reasoning-planning misalignment mainly through autonomous-driving domain alignment with 3M Visual Question Answering (VQA) and subsequent reasoning supervision. Rather than relying on implicit alignment from auxiliary tasks or domain data, DriveMA exploits the verifiability of meta-actions to explicitly measure and encourage consistency between the model’s stated high-level decision and its generated trajectory.

3 DriveMA

3.1 Overview

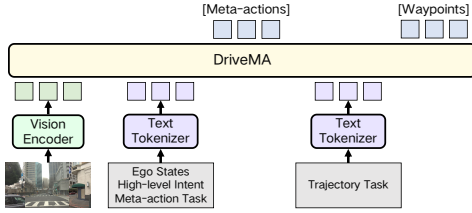


Figure 2: Overview of DriveMA.

DriveMA formulates end-to-end driving planning as a meta-action-guided multi-turn generation problem. Given a driving input x , the model first predicts a compact high-level meta-action m , and then generates future waypoints τ conditioned on both the input and the predicted meta-action:

$$x \rightarrow m \rightarrow \tau.$$

As illustrated in Fig. 2, DriveMA builds on a general vision-language model: visual observations are encoded by the vision encoder, while non-visual inputs and outputs, including trajectory coordinates, are represented through the native text tokenizer.

3.2 Chunk-Based Meta-Action Labeling

Meta-actions are automatically constructed from expert future trajectories with a trajectory-grounded annotation pipeline. Given an expert trajectory τ^* over horizon T , we partition it into non-overlapping chunks of size c and annotate each chunk with a discrete meta-action:

$$m_l^* = \Phi_{\text{ann}}(\tau_{[t_l, t_l+c]}^*).$$

Each meta-action consists of a longitudinal component, selected from *stop*, *decelerate*, *keep*, and *accelerate*, and a lateral component describing coarse maneuver intent such as *going straight*, *turning*, *lane changing*, or *slight shifting*. The annotation function Φ_{ann} combines trajectory-derived geometric cues with dataset-specific thresholds estimated from training trajectories. Additional annotation details are provided in Appendix A. DriveMA uses $c = T$ by default, where a single one-step meta-action summarizes the entire planning horizon.

3.3 Meta-Action-Guided Supervised Training

Before RL, DriveMA uses a two-stage supervised fine-tuning (SFT) pipeline to establish meta-action-guided planning. The first stage adapts the general VLM to driving-domain action understanding [4, 6, 12] and warms up the meta-action prediction task, while the second stage trains the full planning sequence conditioned on expert meta-actions. This staged design separates action-domain adaptation from low-level trajectory generation.

Action-Centric Pretraining. This stage contains two types of supervision. First, we use meta-action prediction data automatically generated from expert trajectories, where the model is trained to predict the expert meta-action from the driving input only, i.e., $x \rightarrow m^*$, without generating future waypoints. This provides a direct warmup for the intermediate decision interface. Second, we incorporate 240K action-related Driving VQA samples from public datasets, selecting samples related to driving intention, action decision, and risk-aware behavior. These samples further align the general VLM with driving-domain perception and decision knowledge.

Meta-Action-Conditioned Planning SFT. After action alignment pretraining, we train the full meta-action-guided planning task, where each sample follows the multi-turn target sequence $x \rightarrow m^* \rightarrow \tau^*$. This stage teaches the model to condition trajectory generation on the expert meta-action decision.

3.4 Turn-Level Credit Assignment Reinforcement Learning

Although supervised fine-tuning introduces the meta-action language interface into planning and improves the model’s planning ability, it may still produce trajectories that are inconsistent with its own high-level decisions. To address this language-action gap, DriveMA exploits the verifiability of meta-actions and introduces a turn-level credit assignment RL framework.

Multi-Turn Rollout. Given a driving input x , we sample G completions from the old policy $\pi_{\theta_{\text{old}}}$, denoted as $o^i = (o_1^i, o_2^i)$ for $i = 1, \dots, G$, where o_1^i is the meta-action prediction turn and o_2^i is the trajectory prediction turn. Let T_j^i denote the token indices of the j -th assistant turn in the i -th completion.

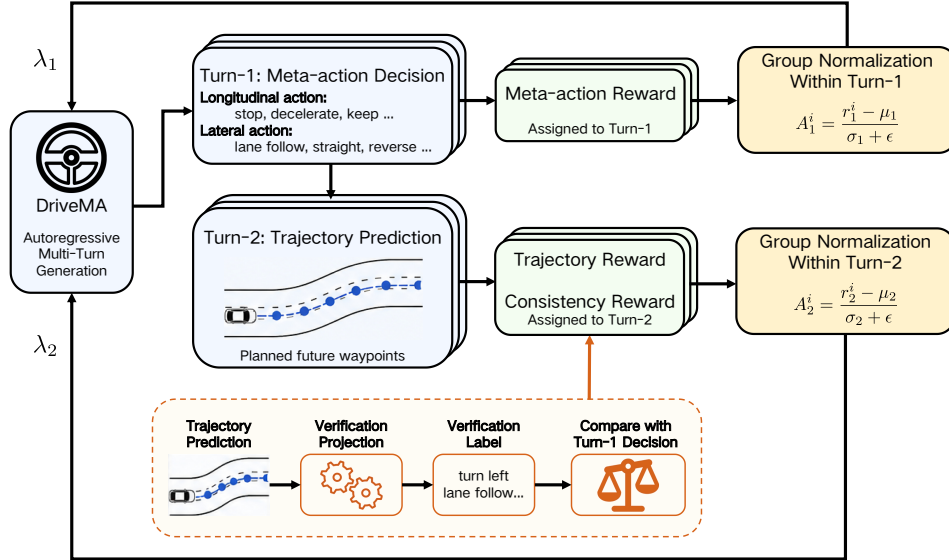


Figure 3: **Turn-level credit assignment RL in DriveMA.** DriveMA assigns meta-action rewards to the decision turn and trajectory/consistency rewards to the planning turn. Rewards are normalized within each turn, enabling dense supervision and precise credit assignment.

Dense Reward Design. DriveMA assigns separate rewards to the two assistant turns:

$$r_1^i = R_{\text{meta}}(o_1^i, M^*), \quad r_2^i = w_{\text{traj}} R_{\text{traj}}(o_2^i, \tau^*) + w_{\text{cons}} R_{\text{cons}}(o_1^i, o_2^i).$$

The first reward evaluates meta-action correctness, while the second combines trajectory quality with language-action consistency. For consistency, we parse the predicted trajectory $\hat{\tau}^i$ from o_2^i and map it to a trajectory-derived verification space with Φ_{ver} . The predicted meta-action $\hat{M}^i$ is also

projected into this space with $\Gamma(\cdot)$:

$$R_{\text{cons}}(o_1^i, o_2^i) = S\left(\Phi_{\text{ver}}(\hat{\tau}^i), \Gamma(\hat{M}^i)\right),$$

where $S(\cdot, \cdot)$ measures agreement in the verification space. This reward encourages the generated trajectory to execute the verifiable motion intent of the predicted meta-action.

Turn-Level Advantage Normalization. Standard GRPO [16, 17] assigns a single sequence-level reward to all generated tokens, which is suboptimal for our multi-turn setting because different turns optimize different objectives. DriveMA instead normalizes rewards separately within each turn:

$$A_j^i = \frac{r_j^i - \mu_j}{\sigma_j + \epsilon}, \quad j \in \{1, 2\},$$

where μ_j and σ_j are computed over the G sampled completions for the j -th turn. Thus, meta-action rewards update only meta-action tokens, while trajectory and consistency rewards update trajectory tokens, yielding more precise credit assignment.

Optimization Objective. The final turn-level credit assignment GRPO objective is:

$$\mathcal{J}_{\text{DriveMA}}(\theta) = \mathbb{E}_{q, \{o^i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}} \left[\frac{1}{G} \sum_{i=1}^G \frac{\sum_{j=1}^2 \lambda_j \frac{1}{|T_j^i|} \sum_{t \in T_j^i} s_{j,t}^i(\theta)}{\sum_{j=1}^2 \lambda_j} \right] - \beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}}),$$

where λ_j controls the relative weight of each turn, and the clipped policy surrogate is $s_{j,t}^i(\theta) = \min(\rho_{i,t}(\theta) A_j^i, \text{clip}(\rho_{i,t}(\theta), 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}) A_j^i)$, with $\rho_{i,t}(\theta) = \pi_{\theta}(o_t^i \mid q, o_{<t}^i) / \pi_{\theta_{\text{old}}}(o_t^i \mid q, o_{<t}^i)$. Here, ϵ_{low} and ϵ_{high} are the lower and upper clipping bounds. The reference policy π_{ref} is the fixed model before RL, and the KL term, weighted by β , is applied as a separate regularizer to constrain policy drift.

4 Experiments

4.1 Experiments Setup

Datasets. We evaluate DriveMA on two end-to-end driving benchmarks: WOD-E2E [7] and NAVSIM [8]. WOD-E2E focuses on challenging long-tail scenarios such as construction zones and safety-critical interactions, and we primarily report its official preference-based metric, RFS. NAVSIM is a planning-oriented autonomous driving benchmark built on OpenScene/nuPlan, and we primarily report PDMS, a closed-loop metric covering safety, progress, and comfort. In addition, we use open-source driving VQA datasets, including WaymoQA [18], IDKB [19], and LingoQA [20], comprising a total of 240K samples, for action-centric pretraining.

Implementation Details. For Waymo, we downsample the provided 10Hz data by a factor of five for training, and DriveMA predicts 5s future trajectories at 1Hz. During RL, we use the official preference subset with 479 expert-scored trajectory samples and instantiate R_{traj} with RFS. For NAVSIM, DriveMA predicts 4s future trajectories at 1Hz, uses the official trainval split with 103K samples, and holds out 18K samples for RL with R_{traj} instantiated by PDMS. For both benchmarks, the model takes only current-frame images from the three front-facing cameras as visual input. DriveMA is built on the Qwen3.5 model family [21], more hyperparameters and prompts are provided in the Appendix B.

4.2 Main Results

WOD-E2E. As shown in Table 1, DriveMA sets a new state of the art on the WOD-E2E. DriveMA-2B and DriveMA-4B achieve **8.060** and **8.079** RFS Overall, respectively, and outperform all previous methods. Notably, DriveMA-2B attains the best results on nearly all key metrics, including **7.251** RFS Spotlight on the official challenging subset and lowest ADE. This result suggests that even a simple language interface can go surprisingly far when it is verifiable and explicitly optimized for language-action alignment. Qualitative comparisons with RAP [26] are in Appendix D.

Table 1: Results on the WOD-E2E. $\uparrow$ indicates higher is better, while $\downarrow$ indicates lower is better. Best results are shown in **bold**, and second-best results are underlined.

Method	RFS Overall $\uparrow$	RFS Spotlight $\uparrow$	ADE@5s $\downarrow$	ADE@3s $\downarrow$
<i>E2E planning methods</i>				
Swin-Trajectory [22]	7.543	6.670	2.814	1.208
DiffusionLTF [23]	7.717	6.414	2.891	1.356
UniPlan [24]	7.780	6.654	2.986	1.308
FROST-Drive [25]	7.856	7.094	3.565	2.537
RAP [26]	8.043	<u>7.204</u>	<u>2.646</u>	1.174
<i>Driving VLA methods</i>				
AutoVLA [4]	7.557	6.944	2.958	1.351
NoRD [13]	7.709	—	—	1.250
HMVLM [9]	7.737	6.727	3.072	1.327
Poutine [11]	7.986	6.893	2.742	1.206
DriveMA-2B	<u>8.060</u>	7.251	2.616	1.154
DriveMA-4B	8.079	7.169	2.670	<u>1.166</u>

NAVSIM Benchmark. Table 2 reports the closed-loop results on NAVSIM. DriveMA achieves competitive performance compared with state-of-the-art end-to-end planners, with DriveMA-4B reaching 91.2 PDMS and outperforming existing VLA-based methods.

Table 2: Results on the NAVSIM v1. All metrics are reported with one decimal place, and higher is better for all metrics. Best results are shown in **bold**, and second-best results are underlined.

Method	PDMS	Collision	Area	Progress	TTC	Comfort
<i>End-to-end planning methods</i>						
TransFuser [27]	84.0	97.7	92.8	79.2	92.8	100.0
Hydra-MDP [28]	86.5	98.3	96.0	78.7	94.6	100.0
TrajHF [29]	86.4	96.6	96.6	84.5	92.1	100.0
DiffusionDriveV2 [30]	<u>91.2</u>	98.3	97.9	87.5	94.8	99.9
RAP [26]	93.8	99.1	<u>98.9</u>	86.0	<u>96.7</u>	100.0
<i>Driving VLA methods</i>						
AutoVLA [4]	89.1	98.4	95.6	81.9	98.0	99.9
DriveVLA-W0 [31]	90.2	98.7	99.1	83.3	95.3	99.3
ReCogDrive [10]	90.8	97.9	97.3	<u>87.3</u>	94.9	100.0
DriveMA-2B	90.5	<u>98.9</u>	97.8	85.3	95.2	100.0
DriveMA-4B	<u>91.2</u>	98.7	97.8	86.7	95.5	99.9

4.3 Ablation Study of Training Strategy

Table 3 analyzes the contribution of each training component. Compared with direct trajectory SFT, meta-action-guided SFT improves RFS Overall from 7.741 to 7.804 and reduces ADE@5s from 3.065 to 2.802, showing that the meta-action interface provides useful but limited gains by itself. Action-centric pretraining further improves RFS Overall to 7.893 and RFS Spotlight to 7.189, suggesting better generalization to challenging scenarios. Turn-level credit assignment further improves over vanilla GRPO under comparable reward supervision, increasing RFS Overall from 7.978 to 8.060 by precisely assigning rewards to their corresponding generation turns.

The reward ablation shows that dense language-action supervision is crucial. Adding R_{cons} substantially improves L-A Consistency from 88.50% to 98.80%, indicating that the consistency reward effectively aligns predicted trajectories with stated meta-actions. As qualitatively shown in Fig. 4, SFT can predict a right-turn meta-action but generate a trajectory that is nearly straight and prematurely stops. The consistency reward mitigates this mismatch by making the trajectory execute the predicted decision. The full reward combination performs best, confirming these objectives are complementary.

Table 3: Ablation study of the training strategy. L-A Consistency is computed in the same way as the language-action consistency reward. ACP denotes Action-Centric Pretraining. All RL variants are initialized from Meta-Action SFT w/ ACP. Vanilla GRPO uses the full reward, but applies them without turn-level credit assignment.

Training Objective	RFS Overall $\uparrow$	RFS Spotlight $\uparrow$	ADE@5s $\downarrow$	L-A Consistency (%) $\uparrow$
<i>Planning interface and SFT baselines</i>				
Direct Trajectory SFT	7.741	7.094	3.065	–
Meta-Action SFT w/o ACP	7.804	6.832	2.802	90.59
Meta-Action SFT w/ ACP	7.893	7.189	2.774	91.31
<i>RL baseline</i>				
Vanilla GRPO w all rewards	7.978	7.087	2.660	95.95
<i>Reward ablation under turn-level credit assignment RL</i>				
R_{traj}	8.009	7.146	2.627	88.50
$R_{\text{traj}} + R_{\text{cons}}$	8.052	7.029	2.674	98.80
$R_{\text{traj}} + R_{\text{meta}}$	8.038	7.018	2.646	91.32
$R_{\text{traj}} + R_{\text{cons}} + R_{\text{meta}}$	8.060	7.251	2.616	98.79



Figure 4: **Qualitative examples of language-action alignment.** SFT predicts a high-level meta-action but generates a trajectory with inconsistent longitudinal and lateral behavior. With the consistency reward, DriveMA generates trajectories that better reflect the predicted meta-action.

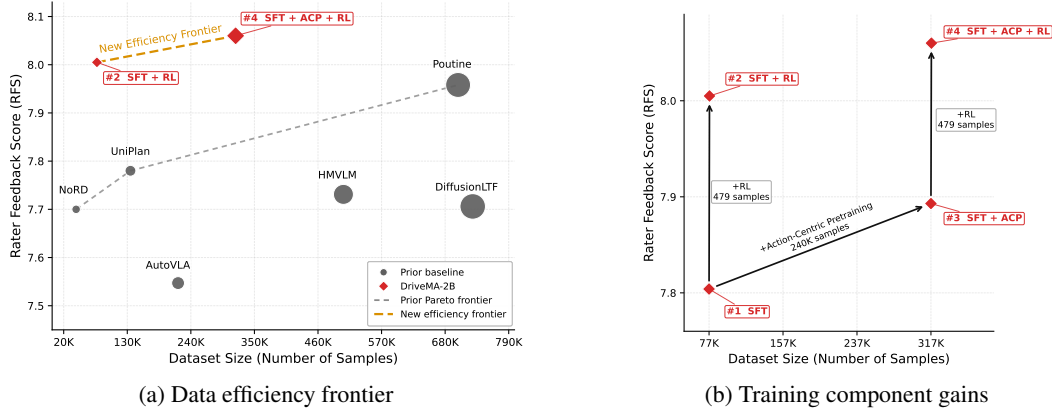


Figure 5: **Data-efficient language-action alignment.** ACP denotes Action-Centric Pretraining. Left: DriveMA shifts the data-efficiency frontier on WOD-E2E, achieving stronger RFS with substantially fewer training samples than prior methods. Dataset sizes of prior methods are taken from NoRD [13]. Right: the gains from RL and ACP are decomposed over DriveMA-2B variants.

4.4 Data-Efficient Language-Action Alignment

Fig. 5 further analyzes the data efficiency of DriveMA on WOD-E2E. Compared with existing methods, DriveMA establishes a new efficiency frontier: with only 77K planning samples and 479 preference samples, DriveMA-2B already reaches 8.005 RFS, surpassing prior VLA baselines trained with substantially larger data. This improvement mainly comes from turn-level credit assignment RL, which effectively exploits limited preference supervision to strengthen language-action alignment. Notably, adding only 479 RL samples improves direct SFT from 7.804 to 8.005 RFS, showing that verifiable meta-actions enable stable and sample-efficient reward optimization. Action-Centric Pretraining (ACP) further improves the model by injecting driving-domain action knowledge, and its combination with RL achieves 8.060 RFS using only 77K planning samples, 240K action-centric

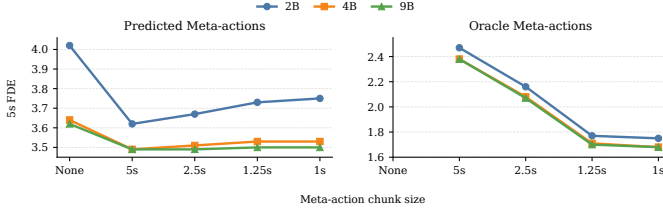


Figure 6: Meta-action granularity analysis on the WOD-E2E validation subset. The plot reports 5s FDE under predicted and oracle meta-action settings across model scales.

Table 4: Complexity of longitudinal action patterns under different chunk sizes in the WOD-E2E training set.

Size	Obs.	Entropy
5s	4	1.761
2.5s	14	2.871
1.25s	122	4.453
1s	221	5.100

pretraining samples, and 479 preference samples. These results suggest that DriveMA does not rely on scaling up supervision alone; instead, verifiable meta-actions allow limited high-quality feedback to be converted into substantial planning gains.

4.5 Meta-Action Granularity Analysis

We analyze the temporal granularity of meta-actions on the WOD-E2E validation subset under supervised fine-tuning. Specifically, we vary the longitudinal chunk size from a single 5s action to finer multi-step actions, while keeping the lateral component as one 5s action. We report 5s Final Displacement Error (FDE) for 2B, 4B, and 9B models under both predicted and oracle meta-action settings, where oracle meta-actions are replaced with ground-truth labels.

As shown in Fig. 6, finer-grained meta-actions improve oracle performance, indicating that they can provide more informative guidance when given correctly. However, this benefit does not transfer to the predicted setting. Predicted performance saturates around the 5s one-step meta-action, and scaling from 4B to 9B brings limited additional gain. This is because finer chunks substantially enlarge the decision space: Table 4 shows that observed longitudinal action patterns increase from 4 to 221 as the chunk size decreases from 5s to 1s. These results suggest that, before RL alignment, the key bottleneck is not only the expressiveness of the meta-action interface but also its predictability. We therefore use one-step meta-actions as DriveMA’s default interface; this analysis also motivates focusing the main evaluation on 2B and 4B variants rather than further backbone scaling.

5 Conclusion

In this paper, we show that verifiability is a key property for turning language into an actionable interface in Driving VLAs. Rather than treating meta-action as a new action vocabulary, DriveMA uses it as a simple instance of a verifiable intermediate language interface. With trajectory-grounded annotation, action-centric pretraining, and turn-level credit assignment RL, DriveMA explicitly aligns high-level language decisions with low-level trajectory planning. Despite the simplicity of this interface, DriveMA achieves a new state of the art on WOD-E2E, competitive performance on NAVSIM, and strong data efficiency with limited preference supervision. These results suggest that future Driving VLAs should not only design more expressive language interfaces, but also make intermediate language states verifiable and optimizable for language-action alignment.

6 Limitations

DriveMA currently exploits verifiability through meta-actions, whose consistency reward relies on projecting generated trajectories into a trajectory-derived verification space. Extending this idea to other language interfaces, such as richer reasoning traces or structured semantic plans, requires interface-specific verification and reward design. In addition, automatically constructed meta-action labels depend on discretization rules, thresholds, and fine-grained lateral disambiguation, which may introduce noise for ambiguous cases near decision boundaries. Future work can explore richer yet still verifiable interfaces that preserve the optimization benefits of meta-actions while improving expressiveness.

References

- [1] H. Shao, Y. Hu, L. Wang, G. Song, S. L. Waslander, Y. Liu, and H. Li. Lmdrive: Closed-loop end-to-end driving with large language models. In *CVPR*, 2024.
- [2] C. Sima, K. Renz, K. Chitta, L. Chen, H. Zhang, C. Xie, J. Beißwenger, P. Luo, A. Geiger, and H. Li. Drivelm: Driving with graph visual question answering. In *ECCV*, 2024.
- [3] X. Tian, J. Gu, B. Li, Y. Liu, Z. Zhao, Y. Wang, K. Zhan, P. Jia, X. Lang, and H. Zhao. Drivelm: The convergence of autonomous driving and large vision-language models. In *CoRL*, 2024.
- [4] Z. Zhou, T. Cai, S. Z. Zhao, Y. Zhang, Z. Huang, B. Zhou, and J. Ma. Autovla: A vision-language-action model for end-to-end autonomous driving with adaptive reasoning and reinforcement fine-tuning. In *NeurIPS*, 2025.
- [5] K. Renz, L. Chen, E. Arani, and O. Sinavski. Simlingo: Vision-only closed-loop autonomous driving with language-action alignment. In *CVPR*, 2025.
- [6] Y. Li, M. Tian, D. Zhu, J. Zhu, Z. Lin, Z. Xiong, and X. Zhao. Drive-r1: Bridging reasoning and planning in vlms for autonomous driving with reinforcement learning, 2025.
- [7] R. Xu, H. Lin, W. Jeon, H. Feng, Y. Zou, L. Sun, J. Gorman, E. Tolstaya, S. Tang, B. White, et al. Wod-e2e: Waymo open dataset for end-to-end driving in challenging long-tail scenarios, 2025.
- [8] D. Dauner, M. Hallgarten, T. Li, X. Weng, Z. Huang, Z. Yang, H. Li, I. Gilitschenski, B. Ivanovic, M. Pavone, et al. Navsim: Data-driven non-reactive autonomous vehicle simulation and benchmarking. In *NeurIPS*, 2024.
- [9] D. Wang, Y. Song, Z. He, K. Chen, X. Pan, L. Deng, and W. Gu. Hmvlm: Multistage reasoning-enhanced vision-language model for long-tailed driving scenarios, 2025.
- [10] Y. Li, K. Xiong, X. Guo, F. Li, S. Yan, G. Xu, L. Zhou, L. Chen, H. Sun, B. Wang, K. Ma, G. Chen, H. Ye, W. Liu, and X. Wang. Recogdrive: A reinforced cognitive framework for end-to-end autonomous driving. In *ICLR*, 2026.
- [11] L. Rowe, R. de Schaetzen, R. Girgis, C. Pal, and L. Paull. Poutine: Vision-language-trajectory pre-training and reinforcement learning post-training enable robust end-to-end autonomous driving, 2025.
- [12] W. Zheng, X. Mao, N. Ye, P. Li, K. Zhan, X. Lang, and H. Zhao. Driveagent-r1: Advancing VLM-based autonomous driving with active perception and hybrid thinking. In *ICLR*, 2026.
- [13] I. Rawal, S. Gupta, Y. Hu, and W. Zhan. Nord: A data-efficient vision-language-action model that drives without reasoning. In *CVPR*, 2026.
- [14] E. Cui, W. Wang, Z. Li, J. Xie, H. Zou, H. Deng, G. Luo, L. Lu, X. Zhu, and J. Dai. Drivelm: aligning multi-modal large language models with behavioral planning states for autonomous driving. *Visual Intelligence*, 2025.
- [15] X. Wang, Q. Liu, W. Ding, Z. Yang, W. Li, C. Liu, B. Li, K. Zhan, X. Lang, and W. Chen. Unifying language-action understanding and generation for autonomous driving, 2026.
- [16] D. Guo, D. Yang, H. Zhang, J. Song, P. Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi, X. Zhang, X. Yu, Y. Wu, Z. F. Wu, Z. Gou, Z. Shao, Z. Li, Z. Gao, A. Liu, B. Xue, B. Wang, B. Wu, B. Feng, C. Lu, C. Zhao, C. Deng, C. Ruan, D. Dai, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Xu, H. Ding, H. Gao, H. Qu, H. Li, J. Guo, J. Li, J. Chen, J. Yuan, J. Tu, J. Qiu, J. Li, J. L. Cai, J. Ni, J. Liang, J. Chen, K. Dong, K. Hu, K. You, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Zhao, L. Wang, L. Zhang, L. Xu,

- L. Xia, M. Zhang, M. Zhang, M. Tang, M. Zhou, M. Li, M. Wang, M. Li, N. Tian, P. Huang, P. Zhang, Q. Wang, Q. Chen, Q. Du, R. Ge, R. Zhang, R. Pan, R. Wang, R. J. Chen, R. L. Jin, R. Chen, S. Lu, S. Zhou, S. Chen, S. Ye, S. Wang, S. Yu, S. Zhou, S. Pan, S. S. Li, S. Zhou, S. Wu, T. Yun, T. Pei, T. Sun, T. Wang, W. Zeng, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, W. L. Xiao, W. An, X. Liu, X. Wang, X. Chen, X. Nie, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yang, X. Li, X. Su, X. Lin, X. Q. Li, X. Jin, X. Shen, X. Chen, X. Sun, X. Wang, X. Song, X. Zhou, X. Wang, X. Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. Zhang, Y. Xu, Y. Li, Y. Zhao, Y. Sun, Y. Wang, Y. Yu, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Ou, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Xiong, Y. Luo, Y. You, Y. Liu, Y. Zhou, Y. X. Zhu, Y. Huang, Y. Li, Y. Zheng, Y. Zhu, Y. Ma, Y. Tang, Y. Zha, Y. Yan, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Xie, Z. Zhang, Z. Hao, Z. Ma, Z. Yan, Z. Wu, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Pan, Z. Huang, Z. Xu, Z. Zhang, and Z. Zhang. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 2025.
- [17] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. Li, Y. Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. 2024.
- [18] S. Yu, S. Lee, N. Kim, J. Shin, J. Park, W. Ryu, R. Jung, and H. Shim. Waymoqa: A multi-view visual question answering dataset for safety-critical reasoning in autonomous driving, 2026.
- [19] Y. Lu, Y. Yao, J. Tu, J. Shao, Y. Ma, and X. Zhu. Can lvlms obtain a driver’s license? a benchmark towards reliable agi for autonomous driving. In *AAAI*, 2025.
- [20] A.-M. Marcu, L. Chen, J. Hünemann, A. Karnsund, B. Hanotte, P. Chidananda, S. Nair, V. Badrinarayanan, A. Kendall, J. Shotton, et al. Lingoqa: Visual question answering for autonomous driving. In *ECCV*, 2024.
- [21] Q. Team. Qwen3.5: Accelerating productivity with native multimodal agents, February 2026. URL <https://qwen.ai/blog?id=qwen3.5>.
- [22] S. Park, G. Shin, J. Song, S. Lee, H. Shon, B. Park, J. Na, H. Jeong, and S. Hwang. Swin-trajectory: Technical report for 2025 waymo vision-based end-to-end driving challenge. *Waymo Open Dataset Workshop (WAD) at CVPR*, 2025.
- [23] L. Nguyen, M. Fauth, B. Jaeger, D. Dauner, M. Igl, A. Geiger, and K. Chitta. Open x-av: Unifying end-to-end autonomous driving datasets. *Waymo Open Dataset Workshop (WAD) at CVPR*, 2025.
- [24] B. Liao, S. Chen, H. Yin, B. Jiang, C. Wang, S. Yan, X. Zhang, X. Li, Y. Zhang, Q. Zhang, et al. Diffusiondrive: Truncated diffusion model for end-to-end autonomous driving. In *CVPR*, 2025.
- [25] Z. Dong, Y. Zhu, Y. Wu, and Y. Sun. Frost-drive: Scalable and efficient end-to-end driving with a frozen vision encoder. In *CVPR*, 2026.
- [26] L. Feng, Y. Gao, E. Zablocki, Q. Li, W. Li, S. Liu, M. Cord, and A. Alahi. RAP: 3d rasterization augmented end-to-end planning. In *ICLR*, 2026.
- [27] K. Chitta, A. Prakash, B. Jaeger, Z. Yu, K. Renz, and A. Geiger. Transfuser: Imitation with transformer-based sensor fusion for autonomous driving. *IEEE TPAMI*, 2023.
- [28] Z. Li, K. Li, S. Wang, S. Lan, Z. Yu, Y. Ji, Z. Li, Z. Zhu, J. Kautz, Z. Wu, et al. Hydra-mdp: End-to-end multimodal planning with multi-target hydra-distillation. 2024.
- [29] D. Li, C. Li, Y. Wang, J. Ren, X. Wen, P. Li, L. Xu, K. Zhan, P. Jia, X. Lang, N. Xu, and H. Zhao. Learning personalized driving styles via reinforcement learning from human feedback, 2025.

- [30] J. Zou, S. Chen, B. Liao, Z. Zheng, Y. Song, L. Zhang, Q. Zhang, W. Liu, and X. Wang. Diffusiondrivev2: Reinforcement learning-constrained truncated diffusion modeling in end-to-end autonomous driving. 2025.
- [31] Y. Li, S. Shang, W. Liu, B. Zhan, H. Wang, Y. Wang, Y. Chen, X. Wang, Y. An, C. Tang, et al. Drivevla-w0: World models amplify data scaling law in autonomous driving. In *ICLR*, 2026.

A Additional Details on Meta-Action Labeling and Verification

DriveMA constructs meta-action labels automatically from expert future trajectories. Each meta-action label consists of a longitudinal component and a lateral component. The longitudinal component contains four labels: `stop`, `decelerate`, `keep`, and `accelerate`, where `stop` also covers waiting behavior. The lateral component contains ten maneuver labels:

<code>straight</code>	<code>lane_follow</code>
<code>left_turn</code>	<code>right_turn</code>
<code>left_lane_change</code>	<code>right_lane_change</code>
<code>left_shift_slightly</code>	<code>right_shift_slightly</code>
<code>reverse</code>	<code>turn_around</code>

The annotation pipeline is trajectory-grounded. Longitudinal labels are derived from future trajectory geometry with deterministic rules, while lateral labels are first resolved by trajectory rules whenever possible and use constrained semantic disambiguation only for visually dependent fine-grained cases. This design provides scalable training labels without human annotation, while preserving a trajectory-derived verification space for consistency reward computation.

Longitudinal annotation. For each trajectory chunk, we compute motion statistics including maximum speed, total displacement, path length, and the slope of the speed profile. Near-static trajectories are identified by clustering low-level motion features over the training set. The resulting low-motion cluster is converted into deterministic thresholds on maximum speed, total displacement, and path length, yielding the rule for `stop`. For the remaining non-static trajectories, we use the speed-profile slope as the acceleration-trend feature. This feature is clustered into three ordered groups, whose boundaries define `decelerate`, `keep`, and `accelerate`. After threshold estimation, all longitudinal labels are assigned by fixed deterministic rules.

Table 5: Longitudinal Meta-Action annotation rules. Thresholds are estimated from training trajectories and then fixed for label construction and consistency verification. Numerical values are rounded to three decimals for readability.

Label	Rule
<code>stop</code>	$v_{\max} \leq 1.333, d_{\text{total}} \leq 1.952, l_{\text{path}} \leq 1.957$
<code>decelerate</code>	$k_v \leq -0.448$
<code>keep</code>	$-0.448 < k_v < 0.429$
<code>accelerate</code>	$k_v \geq 0.429$

Here, $v_{\max}$ denotes the maximum speed in meters per second, d_{total} denotes total displacement in meters, l_{path} denotes path length in meters, and k_v denotes the slope of the speed profile over time. A chunk is labeled as `stop` only when all three low-motion conditions are satisfied. Otherwise, the label is determined by the acceleration-trend rule. This gives a clustering-to-rule procedure: clustering discovers dataset-specific motion boundaries, while the final annotation remains deterministic and interpretable.

Lateral annotation. The lateral label is obtained with a trajectory-first pipeline. For each trajectory chunk, we compute geometric features including net heading change, maximum lateral displacement, final lateral displacement, path length, and motion direction. We denote the net heading change by $\Delta\psi$, the maximum lateral displacement by $d_{\text{lat}}^{\max}$, and the final lateral displacement by $d_{\text{lat}}^{\text{end}}$.

Trajectory rules first identify geometrically clear cases. Near-straight trajectories are detected using heading and lateral-displacement thresholds. Within this near-straight group, `straight`, `left_shift_slightly`, and `right_shift_slightly` are directly assigned according to final lateral displacement. Thus, slight-shift labels are rule-resolved fine-grained labels, rather than left- or right-oriented semantic groups. Backward motion is used to detect `reverse` when the trajectory is

not near-static. For non-straight trajectories, the sign of heading change and final lateral displacement determines left- or right-oriented maneuver groups. The main trajectory-based lateral rules are summarized in Table 6.

Table 6: Lateral Meta-Action rule groups. Trajectory rules directly assign near-straight and slight-shift maneuvers, and produce coarse maneuver groups for visually dependent fine-grained cases.

Case	Rule
near-straight group	$ \Delta\psi \leq 15^\circ$ and $d_{\text{lat}}^{\text{max}} < 1.5$ m
left_shift_slightly	near-straight group and $d_{\text{lat}}^{\text{end}} > 0.75$ m
right_shift_slightly	near-straight group and $d_{\text{lat}}^{\text{end}} < -0.75$ m
straight	near-straight group and $ d_{\text{lat}}^{\text{end}} \leq 0.75$ m
left-oriented group	$\Delta\psi > 15^\circ$ and $d_{\text{lat}}^{\text{end}} > 0$
right-oriented group	$\Delta\psi < -15^\circ$ and $d_{\text{lat}}^{\text{end}} < 0$

Fine-grained lateral disambiguation. Some non-straight lateral maneuvers cannot be reliably distinguished from ego trajectory geometry alone. For example, a left-oriented trajectory may correspond to `left_turn`, `left_lane_change`, or `lane_follow`, depending on road topology and lane structure. Similarly, right-oriented motion may require visual context to distinguish turning from lane changing or lane following.

For these cases, the trajectory rules first generate a small candidate label set. A constrained visual resolver then selects one label from this candidate set using the front-left, front, and front-right camera images together with structured trajectory information. This step is used only to refine visually dependent lateral labels for supervised training. It is not open-ended action generation: the resolver only chooses from rule-provided candidates, which limits semantic drift and keeps the final label grounded in trajectory evidence.

Verification space for consistency reward. The fine-grained meta-action labels above are used as supervised training targets. For consistency reward computation, DriveMA projects both the predicted meta-action and the generated trajectory into a coarser verification space. This avoids requiring exact agreement on visually dependent fine-grained lateral distinctions, such as turn versus lane change, when such distinctions cannot be reliably inferred from ego trajectory geometry alone.

Formally, let $\Phi_{\text{ver}}(\hat{\tau})$ map a generated trajectory to its trajectory-derived verification label, and let $\Gamma(\hat{M})$ project the predicted meta-action into the same verification space. The consistency reward compares these two projected labels:

$$R_{\text{cons}}(o_1, o_2) = S\left(\Phi_{\text{ver}}(\hat{\tau}), \Gamma(\hat{M})\right),$$

where $S(\cdot, \cdot)$ measures agreement in the verification space.

For the longitudinal component, the verification space is the same four-way label space:

$$\{\text{stop}, \text{decelerate}, \text{keep}, \text{accelerate}\}.$$

For the lateral component, verification is performed in a coarse but rule-aware space:

$$\begin{aligned} \text{straight, lane_follow} &\rightarrow \text{forward}, \\ \text{left_shift_slightly} &\rightarrow \text{left_shift_slightly}, \\ \text{right_shift_slightly} &\rightarrow \text{right_shift_slightly}, \\ \text{left_turn, left_lane_change} &\rightarrow \text{left}, \\ \text{right_turn, right_lane_change} &\rightarrow \text{right}, \\ \text{reverse} &\rightarrow \text{reverse}, \\ \text{turn_around} &\rightarrow \text{turn_around}. \end{aligned}$$

Thus, a generated trajectory is considered laterally consistent if it matches the corresponding verification group of the predicted meta-action. For visually dependent labels such as turn versus

lane change, consistency is evaluated at the coarse direction level. For rule-resolved labels such as `left_shift_slightly` and `right_shift_slightly`, consistency is evaluated directly in their own rule-defined categories. This design avoids requiring exact agreement on visually dependent fine-grained distinctions while still preserving rule-verifiable slight-shift behavior.

Overall, the annotation stage provides semantically meaningful fine-grained meta-action labels, while the verification stage evaluates only the motion properties that can be robustly checked from generated trajectories. This separation keeps the training interface expressive enough for planning supervision and the consistency reward reliable enough for reinforcement learning.

B Training and Inference Hyperparameters

Supervised fine-tuning. DriveMA uses a two-stage supervised fine-tuning (SFT) pipeline, consisting of Action-Centric Pretraining (ACP) and meta-action-conditioned planning SFT. Both stages are trained for 1 epoch with a learning rate of 1×10^{-5} and an effective batch size of 64. We perform full-parameter fine-tuning, including the vision encoder, language model, and multimodal projector. All SFT experiments are conducted on 8 NVIDIA A800 GPUs. The maximum image resolution is controlled by `max_pixels`, which is set to $512 \times 512 = 262,144$ pixels.

Reinforcement learning. For RL training, we use a learning rate of 1×10^{-6} and train for 600 steps. The effective batch size is 64, with `num_generations` set to 8 and `steps_per_generation` set to 1. The KL penalty coefficient is set to $\beta = 0.4$, and the sampling temperature is 1.0. The turn-level credit assignment coefficients are set to $\lambda_1 = 0.2$ and $\lambda_2 = 0.8$. For reward weighting, the trajectory reward has weight 1.0, while the consistency reward has weight 0.5. As in SFT, RL uses full-parameter fine-tuning and sets `max_pixels` to $512 \times 512 = 262,144$.

Inference. During inference, we use the same `max_pixels` setting as training. All reported results are obtained with greedy decoding, and we report the top-1 trajectory prediction.

C Training/Inference Prompt Format

DriveMA uses a two-turn dialogue format for meta-action-guided planning. The first turn predicts the high-level meta-action from visual observations and non-visual driving states. The second turn predicts the future ego trajectory conditioned on the same context and the predicted meta-action. The same prompt template is used during training and inference, except that the target response is only provided during training

```
Two-turn prompt format for DriveMA

Human:
You are an expert driver.

Input:
- 1 frame of multi-view images collected from the ego-vehicle at the present timestep:
  front_left_view: <image>; front_view: <image>;
  front_right_view: <image>
- Current high-level intent: <intent>
- Current acceleration: <current_acceleration>
- Current velocity: <current_velocity>
- 4-second past trajectory (16 steps at 4 Hz): <past_trajectory>
- 4-second past acceleration (16 steps at 4 Hz): <past_acceleration>
- 4-second past velocity (16 steps at 4 Hz): <past_velocity>

Coordinate System Definition:
X-axis: positive forward, negative backward;
Y-axis: positive left, negative right.

Task: Inspect the input and make the decision.
Output format:
longitudinal action: xx, lateral action: xx

Assistant:
longitudinal action: <longitudinal_action>, lateral action: <lateral_action>

Human:
Task: Given the above information, predict the optimal 5-second future trajectory (5 steps at 1 Hz) of the ego vehicle.
```

Output format:
 $[x_1, y_1], [x_2, y_2], [x_3, y_3], [x_4, y_4], [x_5, y_5]$

Assistant:
 $[<x_1>, <y_1>], [<x_2>, <y_2>], [<x_3>, <y_3>], [<x_4>, <y_4>], [<x_5>, <y_5>]$



Figure 7: **Qualitative comparison under static-history green-light scenarios.** We compare DriveMA with RAP in three cases where the ego vehicle has nearly zero past motion, but the current traffic light indicates that it should start moving. DriveMA predicts the meta-action *accelerate* and generates a moving trajectory, while RAP tends to remain close to the static history. These cases suggest that the meta-action interface can help reduce history-dominated planning when the future driving intent changes from the recent motion pattern.

D Qualitative Comparison with the Previous State of the Art

We further provide qualitative comparisons between DriveMA and RAP [26], the previous state-of-the-art method on WOD-E2E. These visualizations are intended to complement the quantitative

results by showing representative planning behaviors in challenging scenarios. In each example, we overlay the predicted trajectories of DriveMA and RAP under the same input scene. The comparisons highlight cases where DriveMA’s predicted meta-action provides an explicit high-level motion intent that is faithfully reflected in the generated trajectory.

Starting from static history at green lights. Fig. 7 shows three representative cases where the ego vehicle has an almost fully static history, while the current traffic light indicates that the vehicle should start moving. This setting is challenging because a planner can easily overfit to the near-zero past motion and extrapolate a stationary future. Across all three cases, DriveMA predicts the meta-action *accelerate* and generates a moving trajectory consistent with the green-light context. In contrast, RAP tends to keep the vehicle nearly stationary, suggesting an over-reliance on historical motion states. These examples qualitatively support the motivation in Fig. 1: introducing a verifiable meta-action interface helps DriveMA convert language-level driving intent into executable trajectory planning, especially when the desired future motion differs from the recent motion history.



Figure 8: **Qualitative comparison on route-intent consistency.** We compare DriveMA with RAP in scenarios where the input intent is *GO_STRAIGHT*, but nearby side roads provide plausible turning opportunities. RAP tends to deviate toward the side road, resulting in trajectories inconsistent with the route-level intent. DriveMA predicts the meta-action *decelerate, straight* and generates trajectories that remain aligned with the intended straight-ahead route.

Route-intent consistency at side intersections. Fig. 8 shows cases where the high-level intent is *GO_STRAIGHT*, while the scene contains nearby side roads or turning opportunities. Such scenarios can be ambiguous for trajectory prediction because local road geometry may suggest a feasible turn even when the route-level intent requires continuing straight. In both examples, RAP generates trajectories that deviate toward the side road, producing a plan inconsistent with the input intent. In contrast, DriveMA predicts the meta-action *decelerate, straight* and keeps the planned trajectory aligned with the through lane. In the first case, DriveMA also slows down near the right-side intersection, producing a conservative straight-ahead plan instead of turning toward the side road. In the

second case, DriveMA continues along the current road despite the visible left-side branch. These examples suggest that the explicit meta-action interface helps DriveMA bind route-level intent to trajectory generation, reducing spurious turning behaviors caused by local intersection geometry.



Figure 9: **Qualitative comparison on trajectory curvature control.** We compare DriveMA with RAP in curved-road scenarios where the planner must accurately control turning magnitude to remain within the drivable corridor. RAP produces overly aggressive turning trajectories, leading to potential road-boundary violation in the first case and lane-center crossing in the second case. DriveMA generates smoother trajectories that better follow the lane geometry.

Trajectory curvature control on curved roads. Fig. 9 shows two cases where accurate control of trajectory curvature is important for staying within the drivable corridor. Although the route intent and coarse maneuver are not ambiguous, RAP predicts overly aggressive turning trajectories. In the first case, RAP bends too sharply toward the road boundary under the overpass, which may lead to unsafe interaction with the curb or roadside structure. In the second case, RAP cuts across the lane center and approaches the opposing lane on a left-curving road. In contrast, DriveMA generates smoother trajectories that better follow the lane geometry: it keeps the vehicle within the current lane in the lane-following case and produces a more moderate left-turn trajectory in the curved-road case. These examples indicate that DriveMA’s improvement is not limited to selecting the correct high-level intent; the meta-action-guided planning process also helps produce geometrically better calibrated trajectories under curved road layouts.

Planning under cone-defined temporary corridors. Fig. 10 shows two construction-zone cases where traffic cones define a temporary drivable corridor that differs from the nominal lane geometry. Such scenarios are challenging because blindly following the original lane marking may lead to unsafe interaction with temporary obstacles, while excessive deviation may violate the intended route. In the first case, DriveMA keeps a straight trajectory with a slight left adjustment, following the corridor formed by cones on both sides, whereas RAP drifts toward the roadside. In the second

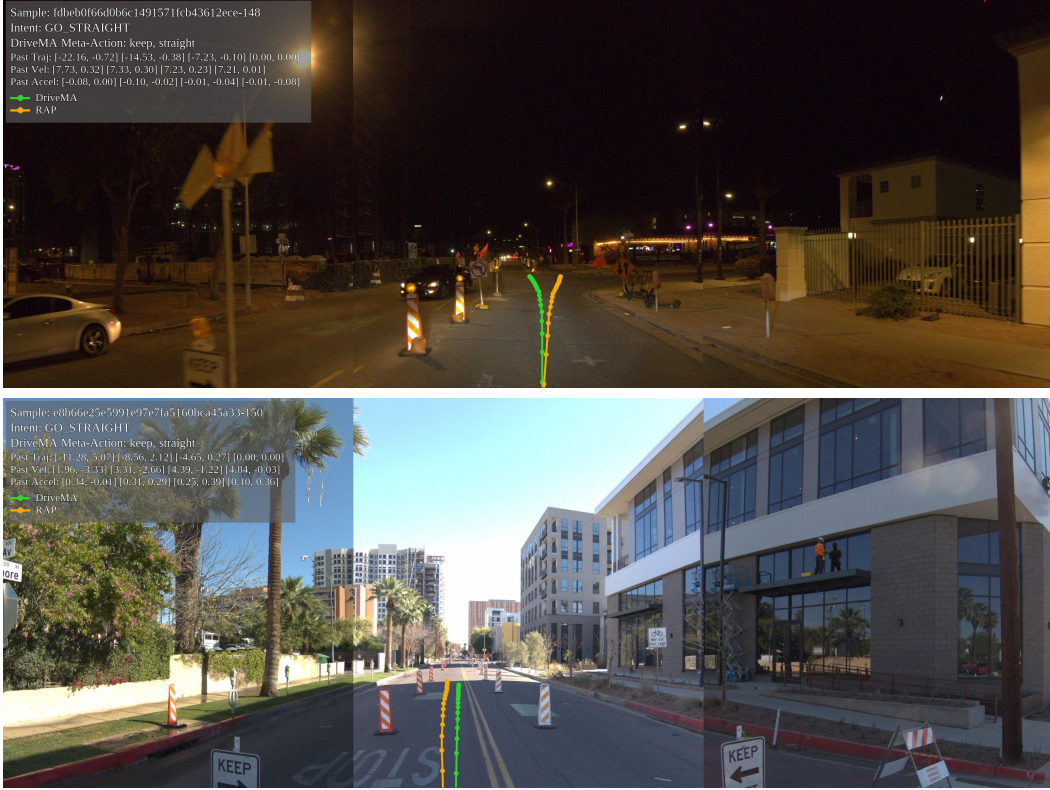


Figure 10: **Qualitative comparison under cone-defined temporary corridors.** We compare DriveMA with RAP in construction-zone scenarios where traffic cones reshape the local drivable corridor. DriveMA generates trajectories that better follow the temporary corridor, while RAP tends to drift toward the roadside or the cone boundary. These cases illustrate DriveMA’s ability to adapt planning to local construction constraints.

case, the cone-defined corridor is shifted near the double-yellow line. Although regular driving would normally avoid crossing the double yellow line, DriveMA produces a more centered trajectory within the temporary corridor, while RAP stays closer to the left-side cones and may risk interacting with them. These examples suggest that DriveMA can better adapt its trajectory to local construction constraints rather than rigidly following nominal lane markings or drifting toward nearby obstacles.

Maneuver-direction errors in complex scenes. Fig. 11 further shows cases where RAP produces trajectories with incorrect maneuver directions under complex scene constraints. These errors are more severe than small geometric deviations, as they directly conflict with the route intent or the locally feasible driving corridor. In the first case, the input intent is *GO_STRAIGHT* and the road ahead is controlled by a red light, where DriveMA predicts *decelerate*, *straight* and plans a conservative straight trajectory. RAP instead deviates to the right, although the scene contains left-guidance markings and no clear rightward route. In the second case, the route intent is *GO_LEFT*; DriveMA predicts a left-turn meta-action and generates a consistent left-turn trajectory, while RAP fails to follow the intended turning direction. In the last case, DriveMA decelerates and keeps straight to handle an oncoming road user, while RAP deviates excessively toward the roadside, likely due to over-conservative obstacle avoidance. These examples suggest that DriveMA is less prone to direction-level planning errors when route intent and local scene constraints must be jointly considered.



Figure 11: **Qualitative comparison on maneuver-direction errors.** We compare DriveMA with RAP in complex scenes involving traffic lights, intersections, and interactive road users. RAP sometimes predicts trajectories with incorrect or over-reactive maneuver directions, such as drifting toward the roadside or failing to follow the intended turn. DriveMA predicts meta-actions consistent with the route intent and generates trajectories that better preserve the intended driving direction under these scene constraints.